

Course outline

Chapter 1. Fundamentals

Chapter 2. Algorithmic paradigms

Chapter 3. Basic data structures

Chapter 4. Tree

Chapter 5. Sorting

Chapter 6. Searching

Chapter 7. Graph

1

Contents

1. Linked List

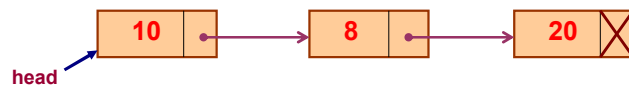
2. Stack

3. Queue

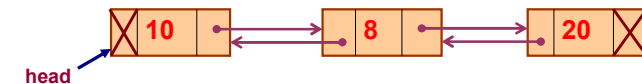
2

1. Linked list

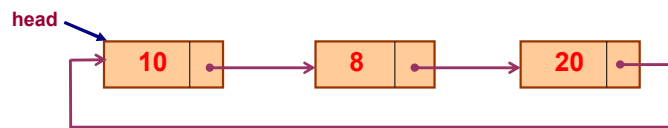
- Singly linked list



- Doubly linked list

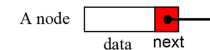
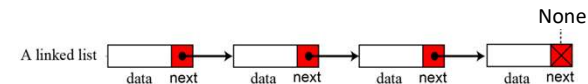


- Circular linked list

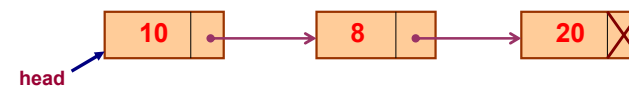


Singly Linked list

- A singly linked list is a sequence of nodes, each node contains 2 parts: **data** (store the value of node) and **next** (store a **reference** to the next node).
- Example: Figure shows a singly linked list contains four nodes:



- Keeping track of a singly linked list:
 - need a reference to the first node of the list (usually called *start*, *head*, etc.)
 - If head is None, the singly linked list is empty



NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Declare singly linked list in Python

List of integer numbers:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

# Create nodes
node1 = Node(7)
node2 = Node(17)
node3 = Node(24)
# Link nodes
node1.next = node2
node2.next = node3
head = node1 # Head points to the first node

# Traverse and print the linked list
current = head
while current is not None:
    print(current.data, end=" -> ")
    current = current.next
print("None")
```

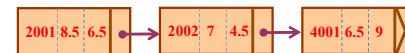
node1.next refers to node2
node2.next refers to node3
node3.next is None
head refers to node1

Output:
7 -> 17 -> 24 -> None

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Declare singly linked list in Python

List of students with data: student's ID, grade of math and physics



```
class Node:
    def __init__(self, student_id, math, physics):
        self.student_id = student_id
        self.math = math
        self.physics = physics
        self.next = None

# Create nodes
node1 = Node("2001", 8.5, 6.5)
node2 = Node("2002", 7, 4.5)
node3 = Node("4001", 6.5, 9)
# Link nodes
node1.next = node2
node2.next = node3
head = node1

class Student:
    def __init__(self, student_id, math, physics):
        self.student_id = student_id
        self.math = math
        self.physics = physics

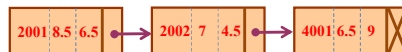
    class Node:
        def __init__(self, data):
            self.data = data # a Student object
            self.next = None # reference to the next node

# Create student data
s1 = Student("2001", 8.5, 6.5)
s2 = Student("2002", 7.0, 4.5)
s3 = Student("4001", 6.5, 9.0)
# Create nodes
node1 = Node(s1)
node2 = Node(s2)
node3 = Node(s3)
# Link the nodes
node1.next = node2
node2.next = node3
head = node1
```

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Declare singly linked list in Python

List of students with data: student's ID, grade of math and physics



```
class Student:
    def __init__(self, student_id, math, physics):
        self.student_id = student_id
        self.math = math
        self.physics = physics
    def __str__(self):
        return f"({self.student_id}, {self.math}, {self.physics})"
```

__str__ is used to represent an object as a string
→ is automatically called when print(cur.data) is executed.
return f"..." is an f-string, used to insert variable values into a string.

Output: ?

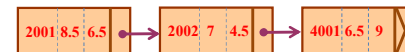
```
class Node:
    def __init__(self, data):
        self.data = data # a Student object
        self.next = None # reference to the next node

# Create student data
s1 = Student("2001", 8.5, 6.5)
s2 = Student("2002", 7.0, 4.5)
s3 = Student("4001", 6.5, 9.0)
# Create nodes
node1 = Node(s1)
node2 = Node(s2)
node3 = Node(s3)
# Link the nodes
node1.next = node2
node2.next = node3

# Head of the list
head = node1
def print_list(head):
    cur = head
    while cur is not None:
        print(cur.data, end=" -> ")
        cur = cur.next
    print_list(head)
```

Declare singly linked list in Python

List of students with data: student's ID, grade of math and physics



```
class Student:
    def __init__(self, student_id, math, physics):
        self.student_id = student_id
        self.math = math
        self.physics = physics
    def __str__(self):
        return f"({self.student_id}, {self.math}, {self.physics})"
```

__str__ is used to represent an object as a string
→ is automatically called when print(cur.data) is executed.
return f"..." is an f-string, used to insert variable values into a string.

Output: ?

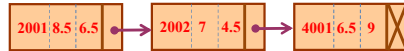
```
class Node:
    def __init__(self, data):
        self.data = data # a Student object
        self.next = None # reference to the next node

# Create student data
s1 = Student("2001", 8.5, 6.5)
s2 = Student("2002", 7.0, 4.5)
s3 = Student("4001", 6.5, 9.0)
# Create nodes
node1 = Node(s1)
node2 = Node(s2)
node3 = Node(s3)
# Link the nodes
node1.next = node2
node2.next = node3

# Head of the list
head = node1
def print_list(head):
    cur = head
    while cur is not None:
        print(cur.data, end=" -> ")
        cur = cur.next
    print_list(head)
```

Declare singly linked list in Python

List of students with data: student's ID, grade of math and physics



```
class Student:
    def __init__(self, student_id, math, physics):
        self.student_id = student_id
        self.math = math
        self.physics = physics
    def __str__(self):
        return f"({self.student_id}, {self.math}, {self.physics})"
    def __repr__(self):
        return "(" + self.student_id + ", " + str(self.math) + ", " + str(self.physics) + ")"
    def __format__(self, format_spec):
        return "{}({}, {}, {})".format(self.student_id, self.math, self.physics)
```

1 `__str__` is used to represent an object as a string
→ is automatically called when `print(cur.data)` is executed.

2 `__repr__` is used to represent an object as a string
→ is automatically called when `print(cur.data)` is executed.

3 `__format__` is used to represent an object as a string
→ is automatically called when `print(cur.data)` is executed.

Declare singly linked list in Python

List of contacts with data: name, phone number



```
class Contact:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
    def __str__(self):
        return f"({self.name}, {self.phone})"
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
# Create contact data
c1 = Contact("Crainic", "0903210433")
c2 = Contact("Michel", "0941341043")
c3 = Contact("Gendreau", "0894312098")
# Create nodes
node1 = Node(c1)
node2 = Node(c2)
node3 = Node(c3)
# Link the nodes
node1.next = node2
node2.next = node3
node3.next = None
# Head of the list
head = node1
def print_list(head):
    cur = head
    while cur is not None:
        print(cur.data, end=" -> ")
        cur = cur.next
    print_list(head)
```

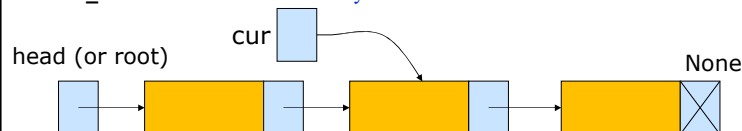
`__str__` is used to represent an object as a string
→ is automatically called when `print(cur.data)` is executed.

`return f"..."` is an f-string, used to insert variable values into a string.

Important elements of singly linked list

- head:** a reference to the first node in the linked list
- None:** indicates that there is no next node
- cur:** a reference to the current node during traversal
- new_node:** a reference to a newly created node

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```



- Create a new node in the list:
`new_node = Node(10)`
- Access to the data of `newNode` :
`new_node.data`
- Access the next node of `new_node`: `new_node.next`
- Update the next reference: `new_node.next = another_node`

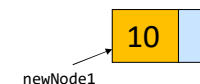
11

Important elements of singly linked list

Example 1: List of integer numbers

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

`newNode1 = Node(10)`

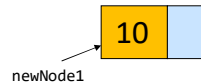


Important elements of singly linked list

Example 1: List of integer numbers

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
newNode1 = Node(10)
newNode1.data = 5
```

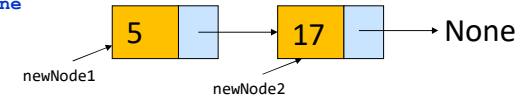


Important elements of singly linked list

Example 1: List of integer numbers

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
newNode1 = Node(10)
newNode1.data = 5
newNode2 = Node(17)
newNode1.next = newNode2
newNode2.next = None
```

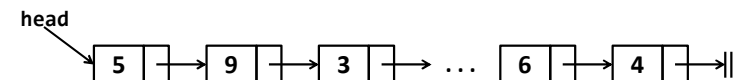


Operations on singly linked Lists

- Traverse the singly linked list
- Insert a node into the singly linked list
- Delete a node from the singly linked list
- Search data in the singly linked list

Assumption

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

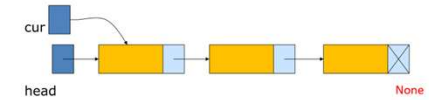


Operations on singly linked Lists

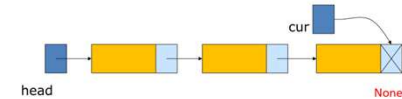
- **Traverse the singly linked list**
- Insert a node into the singly linked list
- Delete a node from the singly linked list
- Search data in the singly linked list

Traversing a singly linked list

```
def print_list(head):  
    cur = head  
    while cur is not None:  
        print(cur.data, end=" -> ")  
        cur = cur.next  
    print("None")
```



- Change the value of the pointer **cur**
- Finish to browse the list when the None value is encountered



```
def print_list(head):  
    cur = head  
    while cur is not None:  
        print(cur.data, end=" " if cur.next is None else " -> ")  
        cur = cur.next
```

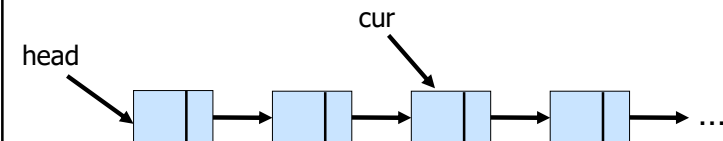
Operations on singly linked lists

- Traverse the singly linked list
- **Insert a node into the singly linked list**
- Delete a node from the singly linked list
- Search data in the singly linked list

Operations on singly linked list: Insertion

Insert a new node :

- At the beginning of the list
- After the position pointed by the pointer cur
- Before the position pointed by the pointer cur
- At the end of the list

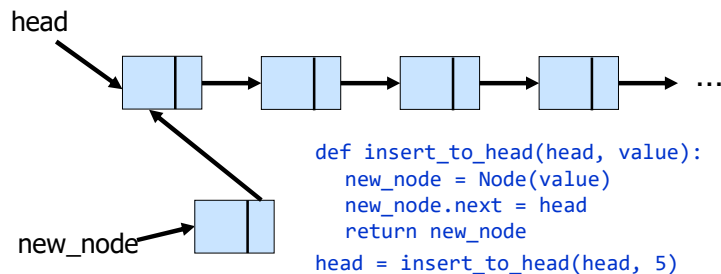


Operations on singly linked list: Insertion

Insert a new node:

- At the beginning of the list

```
<create a new node new_node>
new_node.next = head
head = new_node
```



Exercise 1 on hustack

Read the input data:

- The first line contains n , the number of elements.
- The next line contains n integers.

Each time an integer is read, insert it at the beginning of the linked list by using the function `insert_to_head(head, value)`.

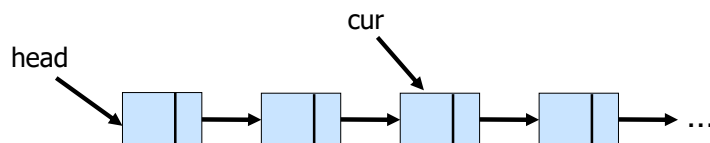
Output: Print all numbers stored in the linked list, separated by spaces.

Chapter 3. Insert the beginning of the singly linked list

Operations on singly linked list: Insertion

Insert a new node :

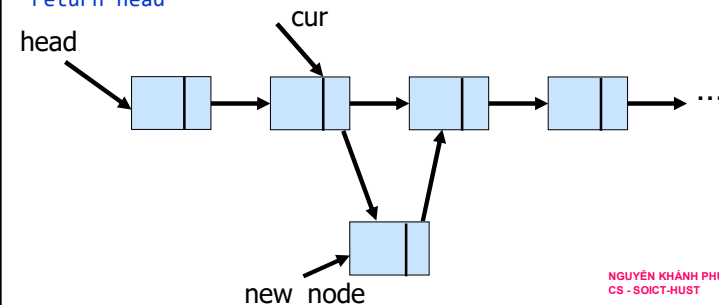
- At the beginning of the list
- After the position pointed by the pointer cur**
- Before the position pointed by the pointer cur
- At the end of the list



Operations on singly linked list: Insertion

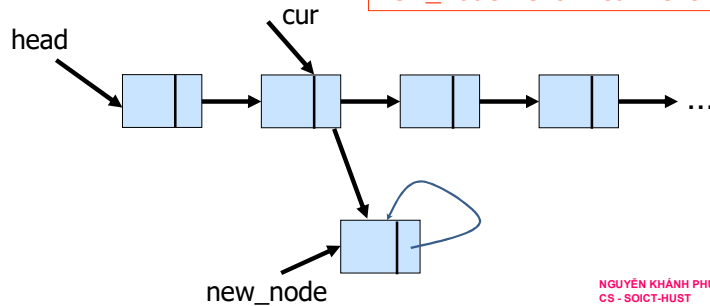
```
def insert_after_node(head, cur, value):
    if cur is None:
        return head
    new_node = Node(value)
    new_node.next = cur.next
    cur.next = new_node

    return head
```



Operations on singly linked list: Insertion

// wrong implementation:
cur.next = new_node
new_node.next = cur.next

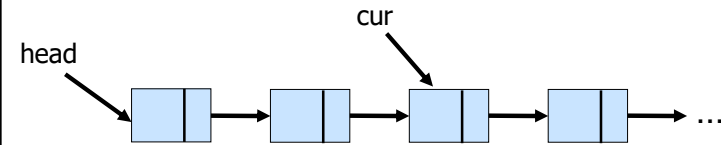


NGUYỄN KHÁNH PHƯƠNG 25
CS - SOICT-HUST

Operations on singly linked list: Insertion

Insert a new node :

- At the beginning of the list
- After the position pointed by the pointer cur
- **Before the position pointed by the pointer cur**
- At the end of the list



NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

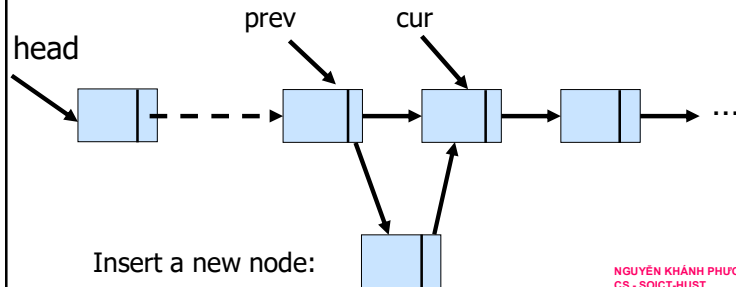
Operations on singly linked list: Insertion

Insert a new node **before the node pointed by the pointer cur**

```
<create a new node new_node>;
<Determine pointer prev pointed
to the predecessor of cur>;
prev->next = new_node;
new_node->next = cur;
```

?? List does not have any node yet

?? cur is the first node in the list



Insert a new node:

NGUYỄN KHÁNH PHƯƠNG 27
CS - SOICT-HUST

Operations on singly linked list: Insertion

Insert a new node **before the node pointed by the pointer cur**

```
<create a new node new_node>
<Determine pointer prev pointed
to the predecessor of cur>
prev.next = new_node
new_node.next = cur
```

?? List does not have any node yet

?? cur is the first node in the list

if head is None: #list does not have any node yet
return head
new_node = Node(value)
if cur == head: #cur is the first node in the list
new_node.next = head
return new_node

```
<Determine pointer prev pointed to the predecessor of cur>;
prev.next = new_node
new_node.next = cur
return head
```

def insert_before_node(head, cur, value)

The function return head of the list after insertion

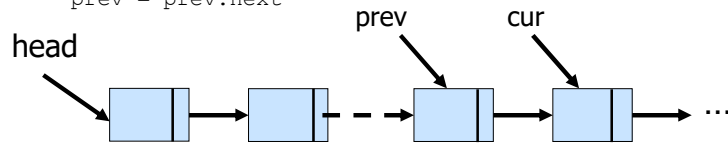
28

Operations on singly linked list: Insertion

Insert a new node **before the node pointed by the pointer cur**

```
<create a new node new_node>
<Determine pointer prev pointed to the predecessor of cur>
prev.next = new_node
new_node.next = cur
```

```
prev = head
while prev.next != cur:
    prev = prev.next
```



```
def insert_before_node(head, cur, value)
```

29

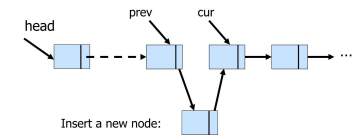
Operations on singly linked list: Insertion

Insert a new node **before the node pointed by the pointer cur**

```
def insert_before_node(head, cur, value):
    if head is None or cur is None:
        return head
    new_node = Node(value)
    # cur is the first node of list
    if head == cur:
        new_node.next = head
        return new_node

    prev = head
    while prev.next != cur:
        prev = prev.next

    prev.next = new_node
    new_node.next = cur
    return head
```



Operations on singly linked list: Insertion

Insert a new node:

- At the beginning
- After the node pointed by cur
- Before the node pointed by cur
- **At the end of the list**



```
def insert_to_tail(head, value):
    new_node = Node(value)
    if head is None:
        return new_node

    cur = head
    while cur.next is not None:
        cur = cur.next

    cur.next = new_node
    return head
```

31

Operations on singly linked Lists

- Traverse the singly linked list
- Insert a node into the singly linked list
- **Delete a node from the singly linked list**
- Search data in the singly linked list

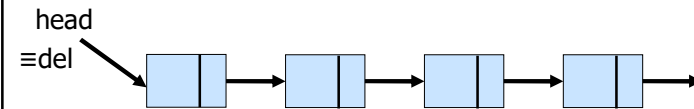
Operations on singly linked lists: Deletion

- **Delete a node**
- Delete all nodes of the list

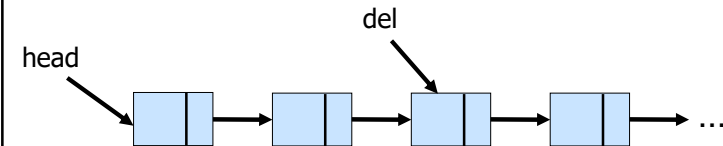
Operations on singly linked lists: Deletion

Delete a node:

- The first node of the list



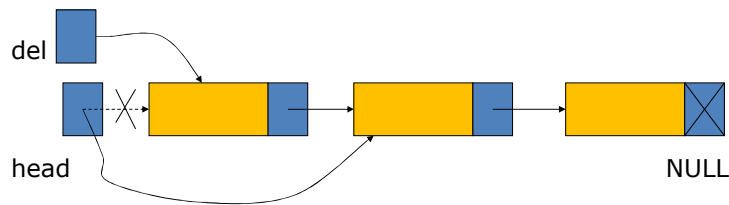
- Middle/last node of the list



Delete the first node of the list

- Delete the node `del` that is currently the first node of the list:

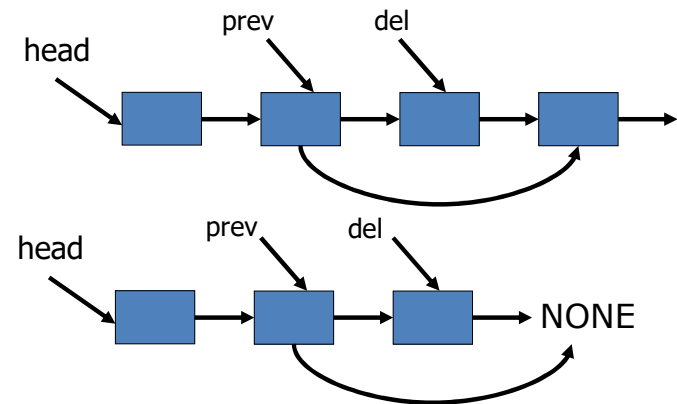
→ `head = del.next;`



Delete the node at the middle/end of the list

Delete node `del` that is currently the middle/last node of the list:

<Determine the pointer `prev` pointed to the previous node of `del`>
`prev.next = del.next;` //modify the link



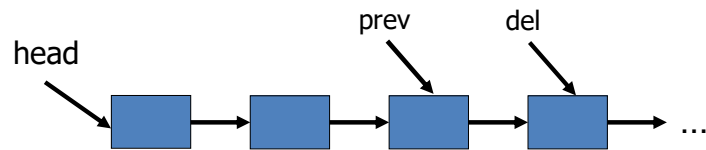
Delete the node at the middle/end of the list

Delete node `del` that is currently the middle/last node of the list:

<Determine the pointer `prev` pointed to the previous node of `del`>

```
prev.next = del.next; //modify the link
```

```
prev = head
while prev->next != del:
    prev = prev.next
```



37

Delete the node referenced by `del_node`

```
def delete_node(head, del_node):
    # Case 1: list is empty
    if head is None:
        return head
    # Case 2: del_node is the first node of list
    if head == del_node:
        head = head.next
        del_node.next = None #optional statement
        return head
    # Case 3: del_node is middle/last node
    prev = head
    while prev.next != del_node:
        prev = prev.next

    # Delete del_node from the list
    prev.next = del_node.next
    del_node.next = None #optional statement

    return head
```

NGUYỄN KHÁNH PHƯƠNG 38
CS - SOICT-HUST

Operations on singly linked lists: Deletion

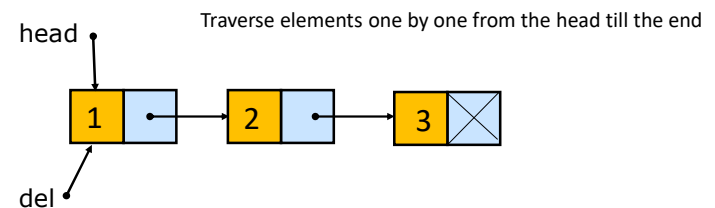
- Delete a node
- **Delete all nodes of the list**

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Freeing all nodes of a list

```
del = head
while del is not None:
```

```
    head = head.next
    del.next = None
    del = head
```

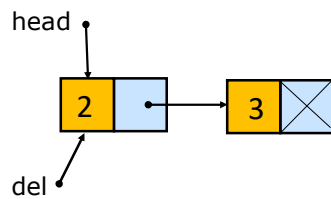


NGUYỄN KHÁNH PHƯƠNG 40
CS - SOICT-HUST

Freeing all nodes of a list

```
del = head
while del is not None:
```

```
    head = head.next
    del.next = None
    del = head
```

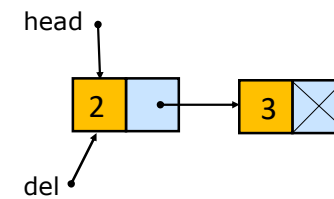


NGUYỄN KHÁNH PHƯƠNG 41
CS - SOICT-HUST

Freeing all nodes of a list

```
del = head
→ while del is not None:
```

```
    head = head.next
    del.next = None
    del = head
```

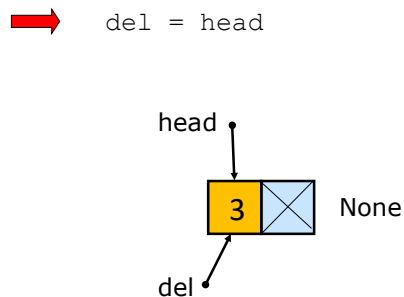


NGUYỄN KHÁNH PHƯƠNG 42
CS - SOICT-HUST

Freeing all nodes of a list

```
del = head
while del is not None:
```

```
    head = head.next
    del.next = None
    del = head
```



NGUYỄN KHÁNH PHƯƠNG 43
CS - SOICT-HUST

Freeing all nodes of a list

- In Python, we usually do not free nodes manually.
- To delete the whole list, we can simply remove the references to the nodes. Python will reclaim the memory automatically.

```
def delete_list(head):
    return None
```

Then call:

```
head = delete_list(head)
```

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Check whether the singly linked list is empty or not

Write the function `is_empty(head)` to check whether the singly linked list is empty or not (the head references to the first node of the list).

The function returns 1 if the list is empty; 0 otherwise

```
def is_empty(head):  
    return head is None
```

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Operations on singly linked Lists

- Traverse the singly linked list
- Insert a node into the singly linked list
- Delete a node from the singly linked list
- **Search data in the singly linked list**

Exercise 2 on hustack

Find the first node in the linked list whose value equals `target`.

```
def search_first(head, target):
```

Return a reference to the first node found.
If the `target` value is not found, return `None`.

47

Build a singly linked list from a list

```
def build_from_list(values):  
    head = None  
    tail = None  
  
    for value in values:  
        new_node = Node(value)  
        if head is None:  
            head = new_node  
            tail = new_node  
        else:  
            tail.next = new_node  
            tail = new_node  
  
    return head
```

48

Exercise 2 on hustack

Write a program that performs the following tasks: Build a linked list whose initial keys are given as the sequence $a_1, a_2, \dots, a_n$. Then perform operations on the list, including: adding one element to the beginning, to the end, before or after a given element in the list, or removing a given element from the list.

Input

Line 1: contains a positive integer n ($1 \leq n \leq 1000$)

Line 2: contains the positive integers $a_1, a_2, \dots, a_n$

The following lines contain commands to manipulate the list, ending with the symbol #. The commands is of the following type:

- **remove k**: remove the first element with key k from the list

Output: Print the sequence of keys of the linked list obtained after performing the given sequence of commands.

```
Input
5
5 4 3 2 1
remove 1
#
Output
5 4 3 2
```

Chapter 3. Singly Linked list with search operation by block

49

Count a number of nodes in a singly linked list

```
def length(head):
```

```
    count = 0
```

```
    cur = head
```

```
    ..... •
```

50

Some other functions

Count the number of times the value **target** appears in the linked list.

```
def count_value(head, target):
```

```
    count = 0
```

```
    cur = head
```

```
    ... •
```

```
    return count
```

51

Exercise 3 on hustack

Write a program that performs the following tasks: Build a linked list whose initial keys are given as the sequence $a_1, a_2, \dots, a_n$. Then perform operations on the list, including: adding one element to the beginning, to the end, before or after a given element in the list, or removing a given element from the list.

Input

Line 1: contains a positive integer n ($1 \leq n \leq 1000$)

Line 2: contains the positive integers $a_1, a_2, \dots, a_n$

The following lines contain commands to manipulate the list, ending with the symbol #. The commands are of the following types:

- **addlast k**: add an element with key k to the end of the list (if k does not already exist)
- **addfirst k**: add an element with key k to the beginning of the list (if k does not already exist)
- **addafter u v**: add an element with key u after the element with key v in the list (if v exists in the list and u does not already exist)
- **addbefore u v**: add an element with key u before the element with key v in the list (if v exists in the list and u does not already exist)
- **remove k**: remove the first element with key k from the list
- **reverse**: reverse the order of the elements in the list (no new nodes may be created; only the links between changed)

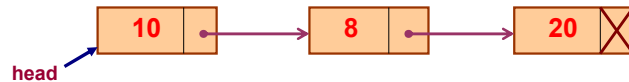
Output: Print the sequence of keys of the linked list obtained after performing the given sequence of commands

```
Input
5
5 4 3 2 1
addlast 3
addlast 10
addfirst 1
addafter 10 4
remove 1
#
Output
5 4 3 2 10
```

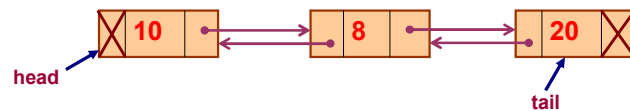
Chapter 3. Singly linked list operations by blocks

1. Linked list

- Singly linked list

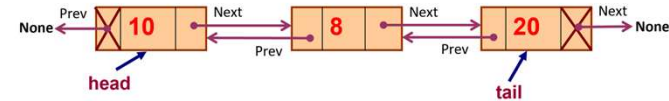


- Doubly linked list



Doubly linked list

- A **Doubly Linked List (DLL)** contains an extra reference, typically called *previous reference*, together with next reference and data which are there in singly linked list

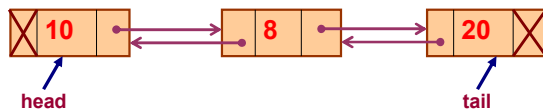


- Each node in a doubly linked list stores:
 - data
 - next**: a reference to the next node
 - prev**: a reference to the previous node
- 2 special nodes: **tail** and **head**
 - head which is the first node, has `head.prev = None`
 - tail which is the last node, has `tail.next = None`
- Basic operations are considered similar as in the singly linked list

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Doubly linked list

- Declare doubly linked list to store integer numbers:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

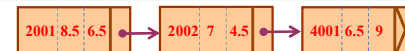
# Create nodes
node1 = Node(10)
node2 = Node(8)
node3 = Node(20)

# Link nodes
node1.next = node2
node2.next = node3
node2.prev = node1
node3.prev = node2

head = node1 # head points to the first node
tail = node3 # tail points to the last node
```

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Declare doubly linked list in Python



List of students with data: student's ID, grade of math and physics

```
class Student:
    def __init__(self, student_id, math, physics):
        self.student_id = student_id
        self.math = math
        self.physics = physics

class Node:
    def __init__(self, data):
        self.data = data # a Student object
        self.next = None # reference to the next node
        self.prev = None # reference to the previous node

# Create student data
s1 = Student("2001", 8.5, 6.5)
s2 = Student("2002", 7.0, 4.5)
s3 = Student("4001", 6.5, 9.0)

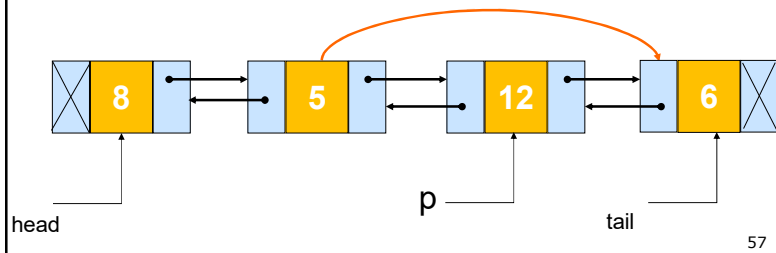
# Create nodes
node1 = Node(s1)
node2 = Node(s2)
node3 = Node(s3)

# Link the nodes
node1.next = node2
node2.next = node3
node2.prev = node1
node3.prev = node2

head = node1
tail = node3
```

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

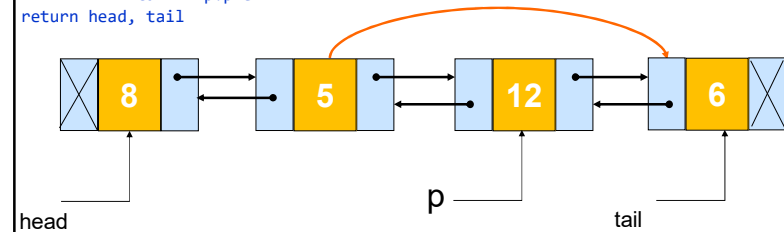
Delete a node referenced by p



57

Delete a node referenced by p

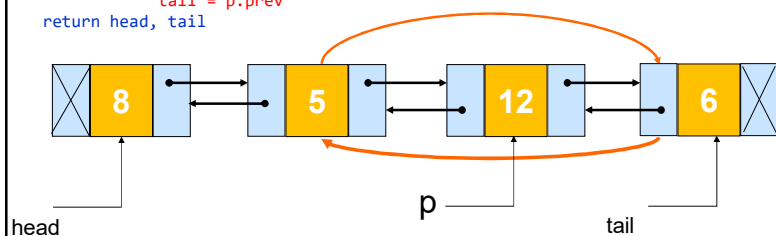
```
def delete_node(head, tail, p):
    if head is None:
        print("Empty list")
    else:
        if p == head: #delete the first node
            head = head.next
        else:
            p.prev.next = p.next
            if p.next is not None:
                p.next.prev = p.prev
            else:
                tail = p.prev
    return head, tail
```



58

Delete a node referenced by p

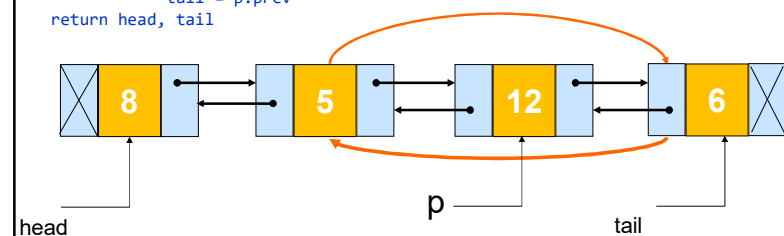
```
def delete_node(head, tail, p):
    if head is None:
        print("Empty list")
    else:
        if p == head: #delete the first node
            head = head.next
        else:
            p.prev.next = p.next
            if p.next is not None:
                p.next.prev = p.prev
            else:
                tail = p.prev
    return head, tail
```



59

Delete a node referenced by p

```
def delete_node(head, tail, p):
    if head is None:
        print("Empty list")
    else:
        if p == head: #delete the first node
            head = head.next
        else:
            p.prev.next = p.next
            if p.next is not None:
                p.next.prev = p.prev
            else:
                tail = p.prev
    return head, tail
```



60

We call: `head, tail = delete_node(head, tail, p)`

Insert a node after the node referenced by p

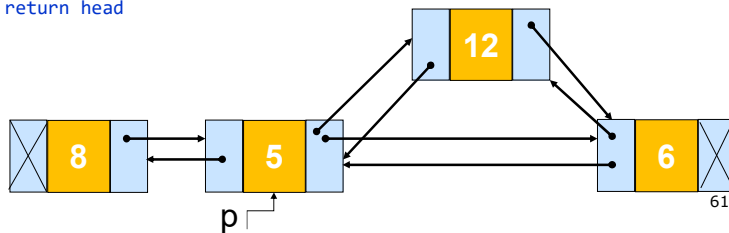
```
def insert_node(X, head, p):
    if head is None: # List is empty
        head = Node(X)
```

```
    else:
        newNode = Node(X)
        newNode.next = p.next
        newNode.prev = p
```

```
        if p.next is not None: #p is not the last node
            p.next.prev = newNode
```

```
        p.next = newNode
```

```
    return head
```



Insert a node after the node referenced by p

```
def insert_node(X, head, tail, p):
    if head is None: # List is empty
        head = Node(X)
        tail = head
```

```
    else:
        newNode = Node(X)
        newNode.next = p.next
        newNode.prev = p
```

```
        if p.next is not None:
            p.next.prev = newNode
```

```
        else:
            tail = newNode
```

```
        p.next = newNode
```

```
    return head, tail
```

Exercise 2: Create a doubly linked list store integer numbers

```
class Node:
```

```
    def __init__(self, number):
        self.number = number
        self.next = None
        self.prev = None
```

```
head = None
```

```
tail = None
```

```
Write the following function:
```

```
Insert a new node p at the end of the list:
```

```
def append_node(p)
```

```
Insert a new node p after a node referenced by variable after
```

```
def insert_node(p, after)
```

```
Delete a node referenced by variable p
```

```
def delete_node(p)
```

```
# Insert a new node p at the end of the list
```

```
def append_node(p):
```

```
    global head, tail
```

```
    if head is None:
```

```
        head = p
```

```
        p.prev = None
```

```
    else:
```

```
        tail.next = p
```

```
        p.prev = tail
```

```
    tail = p
```

```
    p.next = None
```



```

# Insert a new node p after a node pointed by after
def insert_node(p, after):
    global tail
    p.next = after.next
    p.prev = after
    if after.next is not None:
        after.next.prev = p
    else:
        tail = p
    after.next = p

# Delete a node pointed by p
def delete_node(p):
    global head, tail
    if p.prev is None:
        head = p.next
    else:
        p.prev.next = p.next
    if p.next is None:
        tail = p.prev
    else:
        p.next.prev = p.prev

```

```

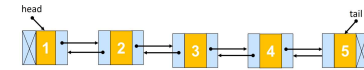
for i in range(1, 6):
    tempnode = Node(i)
    append_node(tempnode)

# print the dll list forward:
print("Traverse the dll list forward")
tempnode = head
while tempnode is not None:
    print(tempnode.number)
    tempnode = tempnode.next

# print the dll list backward:
print("Traverse the dll list backward")
tempnode = tail
while tempnode is not None:
    print(tempnode.number)
    tempnode = tempnode.prev

# destroy the dll list:
while head is not None:
    delete_node(head)

```



Several variants of linked lists

- Some common variants of linked list:
 - Circular Singly Linked Lists
 - Circular Doubly Linked Lists
 - Linked Lists of Lists
- Basic operations on these variants are built similarly to the singly linked list and the doubly linked list that we consider above.

Circular singly linked list

Diagram of a Circular Singly Linked List (CSLL) with 4 nodes containing values 8, 5, 12, and 5. The head pointer points to the first node (8). The last node's next pointer points back to the head, forming a cycle.

```

def printList(head):
    if head is None:
        print("Empty list")
        return
    p = head
    while True:
        print(p.data)
        p = p.next
        if p == head:
            break

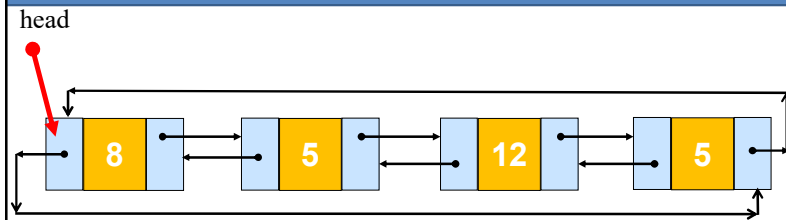
```

```

class Node:
    def __init__(self, data):
        self.number = data
        self.next = None

```

Circular Doubly linked list



```
class Node:
    def __init__(self, data):
        self.number = data
        self.next = None
        self.prev = None
```

Contents

1. Linked List

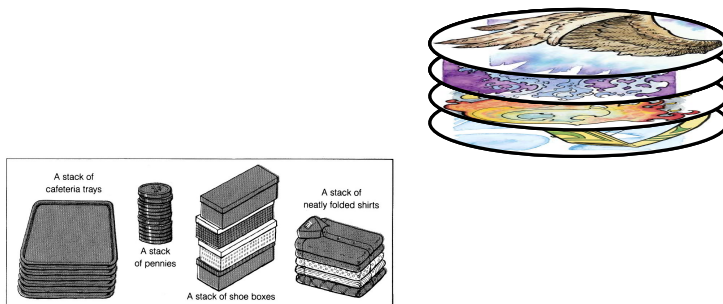
2. Stack

3. Queue

70

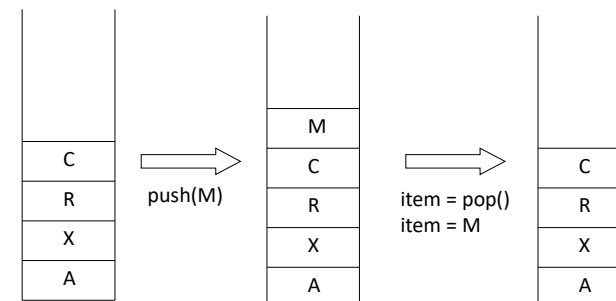
What is a stack?

- A stack is a data structure that only allows items to be inserted and removed at *one end*
 - We call this end the **top** of the stack
 - The other end is called the bottom
- Access to other items in the stack is not allowed
- The last element to be added is the first to be removed (**LIFO**: Last In, First Out)



Operation on stack

- Push*: the operation to place a new item at the top of the stack
- Pop*: the operation to remove the next item from the top of the stack

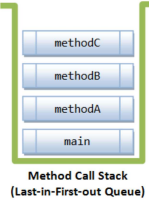


What Are Stacks Used For?

- Real life (Pile of books, Plate trays, etc.)
- More applications related to computer science
 - Program execution stack: Most programming languages use a “call stack” to implement function calling
 - When a method is called, its line number and other useful information are pushed (inserted) on the call stack
 - When a method ends, it is popped (removed) from the call stack and execution restarts at the indicated line number in the method that is now at the top of the stack

```
void methodC()
{
    printf("Enter methodC ");
}
void methodB()
{
    methodC();
}
void methodA()
{
    methodB();
}
void main()
{
    methodA();
}
```

1. main pushed onto call stack, before invoking methodA
2. methodA pushed onto call stack, before invoking methodB
3. methodB pushed onto call stack, before invoking methodC
4. methodC pushed onto call stack, invoked then popped out from the call stack when completes
5. methodB popped out from call stack when completes.
6. methodA popped out from the call stack when completes.
7. main popped out from the call stack when completes. Program exits.



- Evaluating expressions (e.g. $(4/(2-2+3))*(3-4)*2$)

What Are Stacks Used For?

- More applications related to computer science
 - compilers
 - parsing data between delimiters (brackets)
 - Check: each “(”, “{”, or “[” has to pair with “)”, “}”, or “]”

Example:

- correct: `()(()){}{([()])}`
- correct: `((()((()){([()])})`
- incorrect: `)(()){}{([()])}`
- incorrect: `{([ ])}`
- incorrect: `(`

```
//Checking for balanced parentheses is one of the most
//important task of a compiler
int main() {
    for (int i = 0; i < 10; i++) {
        //source code
    }
}
```

- virtual machines
 - manipulating numbers
 - pop 2 numbers off stack, do work (such as add)
 - push result back on stack and repeat
- artificial intelligence
 - finding a path

Example: Reversing a Word

- We can use a stack to reverse the letters in a word.
- How?
- Example: READ →

D
A
E
R

Push(R)
Push(E)
Push(A)
Push(D)

- Read each letter in the word and push it onto the stack

Example: Reversing a Word

- We can use a stack to reverse the letters in a word.
- How?
- Example: READ →

D
A
E
R

Push(R) Pop(D)
Push(E) Pop(A)
Push(A) Pop(E)
Push(D) Pop(R)

- Read each letter in the word and push it onto the stack
- When you reach the end of the word, pop the letters off the stack and print them out

DAE R

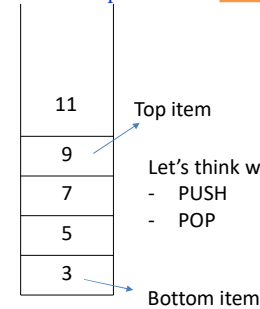
Implementing a Stack

- At least two different ways to implement a stack:
 - array
 - linked list
- Which method to use depends on the application
 - what advantages and disadvantages does each implementation have?

Stack: Array Implementation

- A stack consists of 4 elements: 3, 5, 7, 9.
- If an array is used to implement this stack, what is a good index for the top item?

Is it position 0?	S[0]=9	S[1]=7	S[2]=5	S[3]=3
Is it position 3?	S[0]=3	S[1]=5	S[2]=7	S[3]=9



Let's think when we need do operation:

- PUSH
- POP

The bottom of the stack is at $S[0]$

The top of the stack is at $S[3]$

- pop off of the stack at $S[3]$
- push onto the stack at $S[4]$

Stack: Array Implementation

- If an array is used to implement a stack, the index for the top item is $\text{numItems}-1$ (where numItems is the number of items in the stack)
- Note that push and pop must both work in $O(1)$ time as stacks are usually assumed to be extremely fast
- Implementing a stack using an array is fairly easy:
 - The bottom of the stack is at $S[0]$
 - The top of the stack is at $S[\text{numItems}-1]$
 - push onto the stack at $S[\text{numItems}]$
 - pop off of the stack at $S[\text{numItems}-1]$



Example Stack: Array Implementation

```
class Stack:
    def __init__(self, capacity):
        self.capacity = capacity
        self.arr = [0] * capacity # fixed-size array
        self.top = -1 # stack is empty at the beginning

    def isEmpty(self):
        return self.top == -1

    def isFull(self):
        return self.top == self.capacity - 1

    def push(self, value):
        if self.isFull():
            print("Stack overflow: cannot push", value)
            return
        self.top += 1
        self.arr[self.top] = value

    def pop(self):
        if self.isEmpty():
            print("Stack underflow: stack is empty")
            return None
        value = self.arr[self.top]
        self.top -= 1
        return value

    def top(self):
        if self.isEmpty():
            print("Stack is empty")
            return None
        return self.arr[self.top]

    def size(self):
        return self.top + 1

    def display(self):
        if self.isEmpty():
            print("Stack is empty")
            return
        print("Stack elements from bottom to top:")
        for i in range(self.top + 1):
            print(self.arr[i], end=" ")
        print()
```

```
# Example usage
s = Stack(5)

print("Is stack empty?", s.isEmpty())

s.push(10)
s.push(20)
s.push(30)

s.display()

print("Top element:", s.top())
print("Popped:", s.pop())

s.display()
print("Current size:", s.size())
print("Is stack empty?", s.isEmpty())
```

Example Stack: Array Implementation

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
typedef struct {
    int arr[MAX];
    int top;
} Stack;

// Initialize the stack
void initStack(Stack *s) {
    s->top = -1;
}

// Check whether the stack is empty
int isEmpty(Stack *s) {
    return s->top == -1;
}

// Check whether the stack is full
int isFull(Stack *s) {
    return s->top == MAX - 1;
}

// Push a new integer onto the top of the stack
void push(Stack *s, int value) {
    if (isFull(s)) {
        printf("Stack overflow: cannot push %d\n", value);
        return;
    }
    s->top++;
    s->arr[s->top] = value;
}

// Pop and return the top element from the stack
int pop(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack underflow: stack is empty\n");
        return -1;
    }
    int value = s->arr[s->top];
    s->top--;
    return value;
}

// Return the top element without removing it
int top(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is empty\n");
        return -1;
    }
    return s->arr[s->top];
}

// Return the current number of elements in the stack
int size(Stack *s) {
    return s->top + 1;
}

// Display all elements of the stack from bottom to top
void display(Stack *s) {
    if (isEmpty(s)) {
        printf("Stack is empty\n");
        return;
    }
    printf("Stack elements from bottom to top: ");
    for (int i = 0; i <= s->top; i++) {
        printf("%d ", s->arr[i]);
    }
    printf("\n");
}

int main() {
    Stack s;
    initStack(&s);

    printf("Is stack empty? %s\n", isEmpty(&s) ? "Yes" : "No");

    push(&s, 10);
    push(&s, 20);
    push(&s, 30);

    display(&s);

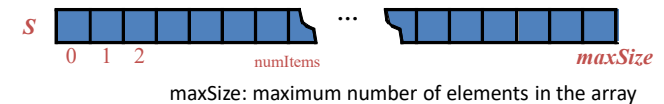
    printf("Top element: %d\n", top(&s));
    printf("Popped: %d\n", pop(&s));
    display(&s);

    printf("Current size: %d\n", size(&s));
    printf("Is stack empty? %s\n", isEmpty(&s) ? "Yes" : "No");

    return 0;
}
```

Array Implementation Summary

- Advantages
 - Easy to implement
 - best performance: push and pop can be performed in $O(1)$ time
- Disadvantage
 - fixed size: the size of the array must be initially specified because
 - The array size must be known when the array is created and is fixed, so that the right amount of memory can be reserved
 - Once the array is full no new items can be inserted
- If the maximum size of the stack is not known (or is much larger than the expected size) a dynamic array can be used
 - But occasionally push will take $O(n)$ time



Stack overflow

- The condition resulting from trying to push an element onto a full stack.

```
if (isFull())
    push(item);
```

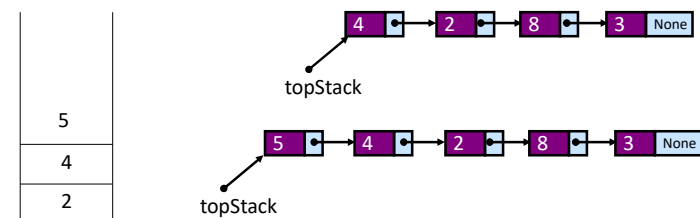
Stack underflow

- The condition resulting from trying to pop an empty stack.

```
if (isEmpty())
    pop(item);
```

Implementing a Stack: using linked list

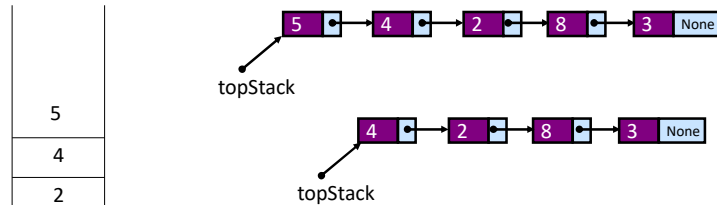
- Store the items in the stack in a linked list
- The top of the stack is the head node, the bottom of the stack is the end of the list
- push* by adding to the front of the list
- pop* by removing from the front of the list



Push operation: need to do the following steps:

- Create new node: allocate memory and assign data for new node
- Link this new node to the top (head) node
- Assign this new node as top (head) node

Implementing a Stack: using linked list



Pop operation: need to do the following steps:

1. Check whether the stack is empty
2. Memorize address of the current top (head) node
3. Memorize data of the current top (head) node
4. Update the top (head) node: the top (head) node now points to its next node
5. Free the old top (head) node
6. Return data of the old top (head) node

If you use Python, you do not need to perform the red-colored line.

Stack: Singly linked list Implementation (Python)

```
1st implementation: use class Node for singly linked list only
class Node:
    def __init__(self, data: int):
        self.data = data
        self.next = None

# Check whether the stack is empty
def isEmpty(topStack):
    return topStack is None

# Push a new integer onto the top of the stack
def push(topStack, value: int):
    newNode = Node(value)
    newNode.next = topStack
    topStack = newNode
    return topStack

# Pop and return the top element from the stack
def pop(topStack):
    if isEmpty(topStack):
        print("Stack underflow: stack is empty")
        return None, topStack
    value = topStack.data
    topStack = topStack.next
    return value, topStack

# Return the top element without removing it
def top(topStack):
    if isEmpty(topStack):
        print("Stack is empty")
        return None
    return topStack.data

# Return the current number of elements in the stack
def size(topStack):
    count = 0
    current = topStack
    while current is not None:
        count += 1
        current = current.next
    return count

# Display all elements of the stack from top to bottom
def display(topStack):
    if isEmpty(topStack):
        print("Stack is empty")
        return
    print("Stack elements from top to bottom:")
    current = topStack
    while current is not None:
        print(current.data)
        current = current.next

# Example usage
topStack = None

print("Is stack empty?", isEmpty(topStack))

topStack = push(topStack, 10)
topStack = push(topStack, 20)
topStack = push(topStack, 30)
display(topStack)
print("Top element:", top(topStack))
value, topStack = pop(topStack)
print("Popped:", value)
display(topStack)
print("Current size:", size(topStack))
print("Is stack empty?", isEmpty(topStack))
```

Stack: Singly linked list Implementation (Python)

```
2nd implementation: use class Node + class Stack
class Node:
    def __init__(self, data: int):
        self.data = data
        self.next = None

class Stack:
    def __init__(self):
        self.topStack = None
        self.count = 0

    # Check whether the stack is empty
    def isEmpty(self):
        return self.topStack is None

    # Push a new integer onto the top of the stack
    def push(self, value: int):
        newNode = Node(value)
        newNode.next = self.topStack
        self.topStack = newNode
        self.count += 1

    # Pop and return the top element from the stack
    def pop(self):
        if self.isEmpty():
            print("Stack underflow: stack is empty")
            return None
        value = self.topStack.data
        self.topStack = self.topStack.next
        self.count -= 1
        return value

    # Return the top element without removing it
    def top(self):
        if self.isEmpty():
            print("Stack is empty")
            return None
        return self.topStack.data

    # Return the current number of elements in the stack
    def size(self):
        return self.count

# Display all elements of the stack from top to bottom
def display(self):
    if self.isEmpty():
        print("Stack is empty")
        return
    current = self.top
    print("Stack elements from top to bottom:")
    while current:
        print(current.data)
        current = current.next

# Example usage
s = Stack()
print("Is stack empty?", s.isEmpty())

s.push(10)
s.push(20)
s.push(30)

s.display()

print("Top element:", s.top())
print("Popped:", s.pop())

s.display()

print("Current size:", s.size())
print("Is stack empty?", s.isEmpty())
```

Stack: Singly linked list Implementation (C)

```
#include <stdio.h>
#include <stdlib.h>

// Define a node of the singly linked list
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Define the stack
typedef struct Stack {
    Node* top;
    int count;
} Stack;

// Initialize the stack
void initStack(Stack* s) {
    s->top = NULL;
    s->count = 0;
}

// Check whether the stack is empty
int isEmpty(Stack* s) {
    return s->top == NULL;
}

// Push a new integer onto the top of the stack
void push(Stack* s, int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = s->top;
    s->top = newNode;
    s->count++;
}

// Pop and return the top element from the stack
int pop(Stack* s) {
    if (isEmpty(s)) {
        printf("Stack underflow: stack is empty\n");
        return -1;
    }
    Node* temp = s->top;
    int value = temp->data;
    s->top = temp->next;
    free(temp);
    s->count--;
    return value;
}

// Return the top element without removing it
int top(Stack* s) {
    if (isEmpty(s)) {
        printf("Stack is empty\n");
        return -1;
    }
    return s->top->data;
}

// Return the current number of elements in the stack
int size(Stack* s) {
    return s->count;
}
```

Stack: Singly linked list Implementation (C)

```
// Display all elements of the stack from top to bottom
void display(Stack* s) {
    if (isEmpty(s)) {
        printf("Stack is empty\n");
        return;
    }
    Node* current = s->top;
    printf("Stack elements from top to bottom:\n");
    while (current != NULL) {
        printf("%d\n", current->data);
        current = current->next;
    }
}

// Free all nodes in the stack
void freeStack(Stack* s) {
    Node* current = s->top;
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    s->top = NULL;
    s->count = 0;
}

int main() {
    Stack s;
    initStack(&s);

    printf("Is stack empty? %s\n", isEmpty(&s) ? "Yes" : "No");

    push(&s, 10);
    push(&s, 20);
    push(&s, 30);

    display(&s);

    printf("Top element: %d\n", top(&s));
    printf("Popped: %d\n", pop(&s));

    display(&s);

    printf("Current size: %d\n", size(&s));
    printf("Is stack empty? %s\n", isEmpty(&s) ? "Yes" : "No");

    freeStack(&s);

    return 0;
}
```

Implementing a Stack: using linked list

- Advantages:
 - always constant time to push or pop an element
 - can grow to an infinite size
- Disadvantages
 - Difficult to implement

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Library in Python: list

list: a data structure that can be used as a stack, where insertion and removal are both performed at the end of the list.

Example:

```
s1 = []                # declare an empty stack
s1.append(x)           # push an element into the stack
s1.pop()              # remove the top element
s1[-1]                # get the top element
len(s1)               # return the number of elements in the stack
len(s1) == 0          # check whether the stack is empty
```

NGUYỄN KHÁNH PHƯƠNG 91
CS - SOICT-HUST

Library in Python: list

list: a data structure that can be used as a stack, where insertion and removal are both performed at the end of the list.

Example:

```
s1 = []
s1.append(10)
s1.append(20)
s1.append(30)

print(s1)
print(s1[-1])

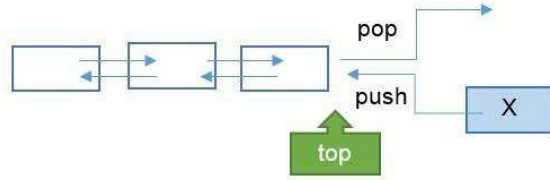
s1.pop()
print(s1)
```

Output:
[10, 20, 30]
30
[10, 20]

NGUYỄN KHÁNH PHƯƠNG 92
CS - SOICT-HUST

Library STL in C++: stack

`#include <stack>`: a list type where insertion and removal are both performed at the end of the list.



Example about declaration:

```
stack<int> s1; //declare variable stack s1 in which type of each element in s1 is int
```

- We can not declare `stack<int> myStack (5)` to initialize a stack with 5 empty elements as in array.
- Similarly, we can not declare `stack<int> myStack (5,100)` to initialize a stack consisting of 5 elements with values 100.

NGUYỄN KHÁNH PHƯƠNG 93
CS - SOICT-HUST

Library STL in C++: stack

Capacity	
size()	Return the number of elements currently in stack
empty()	True/False
Access to an element	
top()	Access to top element of stack
Query	
push(x)	Insert an element with value x on the top of stack. Size of stack will be increased by 1
pop()	Remove the top element from stack
swap(stack s1, stack s2)	Swap elements of stack s1 and s2

NGUYỄN KHÁNH PHƯƠNG 94
CS - SOICT-HUST

Application 1: Parentheses Matching

Check for balanced parentheses in an expression:

Given an expression string expression, write a program to examine whether the pairs and the orders of “{”, “}”, “(”, “)”, “[”, “]” are correct in expression.

For example, the program should print true for expression = “[(){}]{(OO)O}” and false for expression = “[()]”

```
//Checking for balanced parentheses is one of the most
//important task of a compiler

int main() {
    for (int i = 0; i < 10; i++) {
        //source code
    }
}
```

Algorithm:

- 1) Declare a character stack S.
- 2) Now traverse the expression string expression
 - a) If the current character is a starting bracket ('(', '{' or '[') then push it to stack.
 - b) If the current character is a closing bracket (')', '}' or ']') then pop from stack and if the popped character is the matching starting bracket then fine else parenthesis are not balanced.
- 3) After complete traversal, if there is some starting bracket left in stack then “not balanced”

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Application 1: Parentheses Matching

Algorithm ParenMatch(X,n):

Input: Array X consists of n characters, each character could be either parentheses, variable, arithmetic operation, number.

Output: true if parentheses in an expression are balanced

S = stack empty;

for i=0 to n-1 do

if (X[i] is a starting bracket)

push(S, X[i]); // starting bracket ('(', '{' or '[') then push it to stack

else

if (X[i] is a closing bracket) // compare X[i] with the one currently on the top of stack

if isEmpty(S)

return false {can not find pair of brackets}

if (pop(S) not pair with bracket stored in X[i])

return false {error: type of brackets}

if isEmpty(S)

return true {parentheses are balanced}

else return false {there exist a bracket not paired}

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Exercise on hustack: Chapter 3. Check Parenthesis

Given a string containing only characters (,), [,], {, }, }. Write a program that checks whether the string is correct in expression.

Example

- ([] { }) []: correct
- ([] { }) []: incorrect

Input

- One line contains the string (the length of the string is less than or equal to 10^6)

Output

- Write 1 if the sequence is correct, and write 0, otherwise

Example

Input

([[]]){}[]([[]])

Output

1

Application 2: Calculate the value of an expression

Example: Calculate the value of expression: $(4/(2-2+3))*(3-4)*2)$

Node: Priority order of operations from high to low:

1. Exponent ^
2. Multiplication Division * /
3. Addition subtraction + -

The algorithm use 2 stacks:

- Stack S1 stores operands
- Stack S2 stores operators (+, -, *, /, ^) and opening bracket (

Procedure Process:

- Do Pop(S1) twice: Pop off 2 values on the top of S1 from S1, denote these 2 values by **A** and **B**
- Do Pop(S2): Pop off 1 operator on the top of S2 from S2, denote it by **OPT**.
- Do **B OPT A**, the result is stored in **R**
- Push R into S1: do Push(S1, R)

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Application 2: Calculate the value of an expression

Scan the expression from left to right:

- If the current element is *Operand*: push it into stack S1.
- If the current element if *Operator*, denote it by **OPT**:
 - If stack S2 is empty: push operator **OPT** into stack S2, that means: Push(S2, OPT)
 - Else if **OPT** has higher priority than that of top(S2) then push OPT into stack S2, that means Push(S2, OPT)
 - Else if **OPT** has priority lower or equal to that of top(S2) then
 - do Process **Until** operator OPT has higher priority than that of top(S2) OR stack S1 is empty
 - Push(S2, OPT)
- If the current element if *opening bracket*: then push it into stack S2.
- If the current element if *closing bracket*: then
 - do Process ... **Until** found the first opening bracket in S2
 - Pop the opening bracket from S2
- *When finish to scan the expression and Stack S1 is not empty*:
 - do Process **Until** Stack S1 is empty

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Example: Calculate $2 * (5 * (3 + 6)) / 15 - 2$

Element	Operation	Stack S1	Stack S2	Explanation
2	Push 2 into S1	2	Empty	
*	Push * into S2	2	*	
(	Push (into S2	2	(*	
5	Push 5 into S1	5 2	(*	
*	Push * into S2	5 2	(* *	
(	Push (into S2	5 2	(* (*	
3	Push 3 into S1	3 5 2	(* (*	
+	Push + into S2	3 5 2	(* (* +	
6	Push 6 into S1	6 3 5 2	(* (* +	

Scan the expression from left to right:

- If the current element is *Operand*: push it into stack S1.
- If the current element if *Operator*, denote it by **OPT**:
 - If stack S2 is empty: Push(S2, OPT)
 - Else if **OPT** has higher priority than that of top(S2) then Push(S2, OPT)
 - Else if **OPT** has priority lower or equal to that of top(S2) then
 - do Process **Until** operator **OPT** has higher priority than that of top(S2) OR stack S1 is empty
 - Push(S2, OPT)
- If the current element if *opening bracket*: then push it into stack S2.
- If the current element if *closing bracket*: then
 - do Process ... **Until** found the first opening bracket in S2
 - Pop the opening bracket from S2
- *When finish to scan the expression and Stack S1 is not empty*:
 - do Process **Until** Stack S1 is empty

Example: Calculate $2 * (5 * (3 + 6)) / 15 - 2$

Procedure Process:

- Do Pop(S1) twice: Pop off 2 values on the top of S1 from S1, denote these 2 values by **A** and **B**
- Do Pop(S2): Pop off 1 operator on the top of S2 from S2, denote it by **OPT**.
- Do **B OPT A**, the result is stored in **R**
- Push R into S1: do Push(S1, R)

)	Call Process:			Do Process until found the first opening bracket in S2
	Pop 6 and 3 from S1	5 2	+ (* (*	
	Pop + from S2	5 2	(* (*	
	Do 3+6=9	5 2	(* (*	
	Push 9 into S1	9 5 2	(* (*	
	Pop (from S2	9 5 2	* (*	

Example: Calculate $2 * (5 * (3 + 6)) / 15 - 2$

Element	Operation	Stack S1	Stack S2	Explanation
)	Call Process:			Do Process until found the first opening bracket in S2
	Pop 9 and 5 from S1	2	* (*	
	Pop * from S2	2	(*	
	Do 5*9=45	2	(*	
	Push 45 into S1	45 2	(*	
	Pop (from S2		*	
/	Call Process:			/ and * have the same priority
	Pop 45 and 2 from S1		*	Do Process
	Pop * from S2			
	Do 2*45=90			
	Push 90 into S1	90		
	Push / into S2		/	

Scan the expression from left to right:

- If the current element is *Operand*: push it into stack S1.
- If the current element is *Operator*, denote it by **OPT**:
 - If stack S2 is empty: Push(S2, OPT)
 - Else if **OPT** has higher priority than that of top(S2) then Push(S2, OPT)
 - Else if **OPT** has priority lower or equal to that of top(S2) then
 - do Process **Until** operator **OPT** has higher priority than that of top(S2) OR stack S1 is empty
 - Push(S2, OPT)
- If the current element is *opening bracket*: then push it into stack S2.
- If the current element is *closing bracket*: then
 - do Process ... **Until** found the first opening bracket in S2
 - Pop the opening bracket from S2
- When finish to scan the expression and Stack S1 is not empty:
 - do Process **Until** Stack S1 is empty

Procedure Process:

- Do Pop(S1) twice: Pop off 2 values on the top of S1 from S1, denote these 2 values by **A** and **B**
- Do Pop(S2): Pop off 1 operator on the top of S2 from S2, denote it by **OPT**.
- Do **B OPT A**, the result is stored in **R**
- Push R into S1: do Push(S1, R)

Example: Calculate $2 * (5 * (3 + 6)) / 15 - 2$

Element	Operation	Stack S1	Stack S2	Explanation
15	Push 15 into S1	15 90	/	
-	Call Process		/	"-" has lower priority than "/", so do Process
	Pop 15 and 90 from S1			
	Pop / from S2			
	Do 90/15=6			
	Push 6 into S1	6		
-	Push - into S2	6	-	

Scan the expression from left to right:

- If the current element is *Operand*: push it into stack S1.
- If the current element is *Operator*, denote it by **OPT**:
 - If stack S2 is empty: Push(S2, OPT)
 - Else if **OPT** has higher priority than that of top(S2) then Push(S2, OPT)
 - Else if **OPT** has priority lower or equal to that of top(S2) then
 - do Process **Until** operator **OPT** has higher priority than that of top(S2) OR stack S1 is empty
 - Push(S2, OPT)
 - If the current element is *opening bracket*: then push it into stack S2.
 - If the current element is *closing bracket*: then
 - do Process ... **Until** found the first opening bracket in S2
 - Pop the opening bracket from S2
 - When finish to scan the expression and Stack S1 is not empty:
 - do Process **Until** Stack S1 is empty

Procedure Process:

- Do Pop(S1) twice: Pop off 2 values on the top of S1 from S1, denote these 2 values by **A** and **B**
- Do Pop(S2): Pop off 1 operator on the top of S2 from S2, denote it by **OPT**.
- Do **B OPT A**, the result is stored in **R**
- Push R into S1: do Push(S1, R)

Example: Calculate $2 * (5 * (3 + 6)) / 15 - 2$

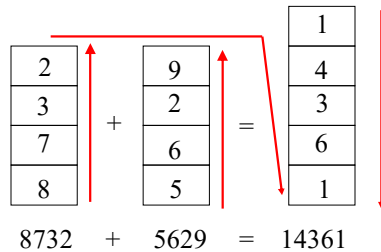
Element	Operation	Stack S1	Stack S2	Explanation
-	Push - into S2	6	-	
2	Push 2 into S1	2 6	-	
	Call Process			Already finish to scan all elements in expression, but S1 is not empty. Therefore, call Process until S2 is empty
	Pop 2 and 6 from S1	EMPTY	-	
	Pop - from S2	EMPTY	EMPTY	
	Do 6-2=4			
	Push 4 into S1	4		

The value of expression = 4

Application 3: Adding two large numbers

- Treat these numbers as strings of numerals, store the numbers corresponding to these numerals on two stacks, and then perform addition by popping numbers from the stacks

123456789012345678901234567890
7890123456789012345



Application 3: Adding two large numbers

```
Read the first number A as a string
Read the second number B as a string
Create three empty stacks: sA, sB, sResult
For each digit in A:
    push it onto sA
For each digit in B:
    push it onto sB
carry = 0
while sA is not empty OR sB is not empty:
    if sA is not empty:
        x = pop(sA)
    else:
        x = 0
    if sB is not empty:
        y = pop(sB)
    else:
        y = 0
    sum = x + y + carry
    push (sum mod 10) onto sResult
    carry = sum div 10
if carry != 0:
    push carry onto sResult
while sResult is not empty:
    output pop(sResult)
```

106

Contents

1. Array
2. Linked List
3. Stack
- 4. Queue**

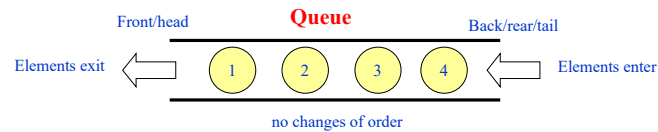
107

What is a Queue?



Queues

- What is a queue?
 - A data structure of ordered items such that items can be inserted only at one end and removed at the other end.



Example: A line at the supermarket

- What can we do with a queue?
 - Enqueue** (insert) – Add an item to the queue (also called as insert)
 - Dequeue** (delete) – Remove an item from the queue (also called as getFront)
- A queue is called a FIFO (First in-First out) data structure.

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Queue

- Terms used in Queue are depicted in the following figure:



NGUYỄN KHÁNH PHƯƠNG
KHMT - SOICT - ĐHBK HN

Queue specification

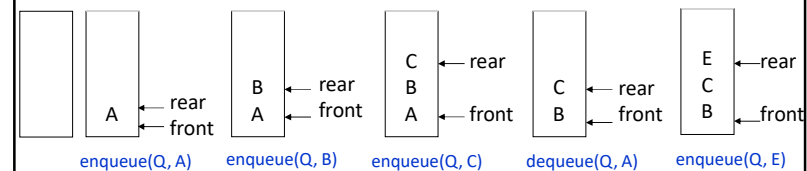
Definitions: (provided by the user)

- maxSize*: Max number of items that might be on the queue
- ItemType*: Data type of the items on the queue

Operations:

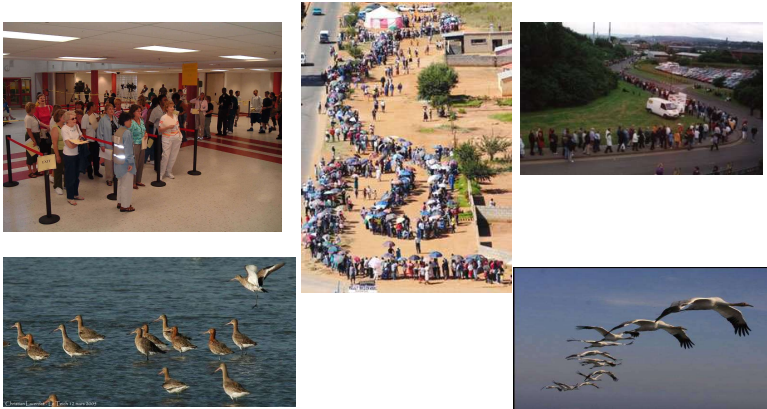
- `Q = init();` initialize empty queue Q
- `isEmpty(Q);` returns "true" if queue Q is empty
- `isFull(Q);` returns "true" if Q is full, indicates that we already use the maximum memory for queue; otherwise returns "false"
- `front(Q);` returns the item that is in front (head) of queue Q or returns error if queue Q is empty.
- `enqueue(Q, x);` inserts item x into the back (rear) of queue Q. If before making insertion, the queue Q is full, then give the notification about that.
- `x = dequeue(Q);` deletes the element at the front (head) of the queue Q, then returns x which is the data of this element. If the queue Q is empty before dequeue, then give the error notification.
- `display(Q);` gives the list of all elements in the queue Q in the order from the front to the back.
- `size(Q);` returns the number of elements currently in the queue Q.

FIFO



NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Queues everywhere!!!!



Queues

- What are some applications of queues?
 - Round-robin scheduling in processors
 - Input/Output processing
 - Queueing of packets for delivery in networks

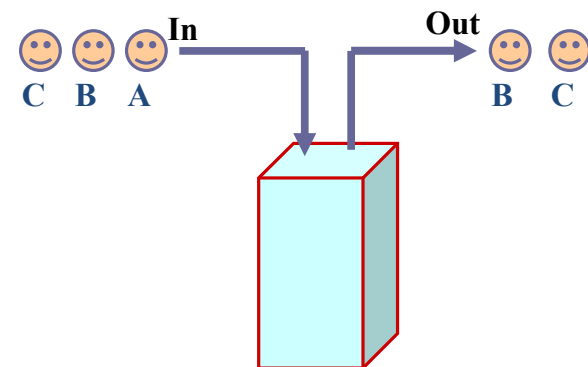
NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Given the sequence of operations on queue Q as following. Determine the output and the data on the queue Q after each operation:

	Operation	Output	Queue Q
1	enqueue (Q, 5)	-	(5)
2	enqueue (Q, 3)	-	(5, 3)
3	dequeue (Q)	5	(3)
4	enqueue (Q, 7)	-	(3, 7)
5	dequeue (Q)	3	(7)
6	front (Q)	7	(7)
7	dequeue (Q)	7	()
8	dequeue (Q)	error	()
9	isEmpty (Q)	true	()
10	size (Q)	0	()
11	enqueue (Q, 9)	-	(9)
12	enqueue (Q, 7)	-	(9, 7)
13	enqueue (Q, 3)	-	(9, 7, 3)
14	enqueue (Q, 5)	-	(9, 7, 3, 5)
15	dequeue (Q)	9	(7, 3, 5)

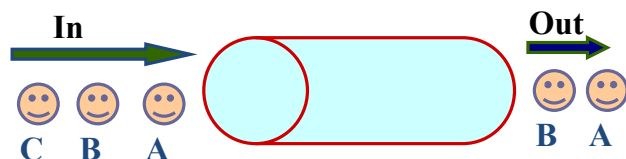
Stack

Data structure with **Last-In First-Out (LIFO)** behavior



Queue

Data structure with **First-In First-Out (FIFO)** behavior

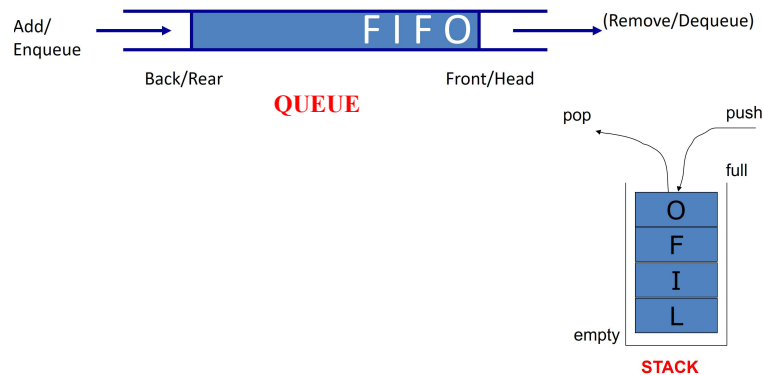


Implementing a Queue

- Just like a stack, we can implement a queue in two ways:
 - Using an array
 - Using a linked list

Implementing a Queue: using Array

- Using an array to implement a queue is significantly harder than using an array to implement a stack. Why?
 - A stack: we add and remove at the same end,
 - A queue: we add to one end and remove from the other.



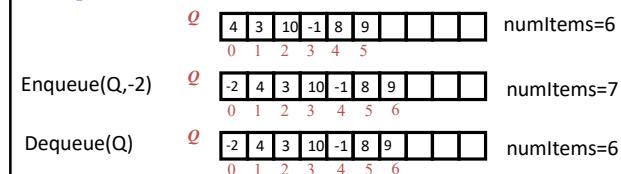
Implementing a Queue: using Array

There are some options for implementing a queue using an array:

- Option 1:
 - Enqueue at $Q[0]$ and shift all of the rest of the items in the array down to make room.
 - Dequeue from $Q[numItems-1]$



Example:



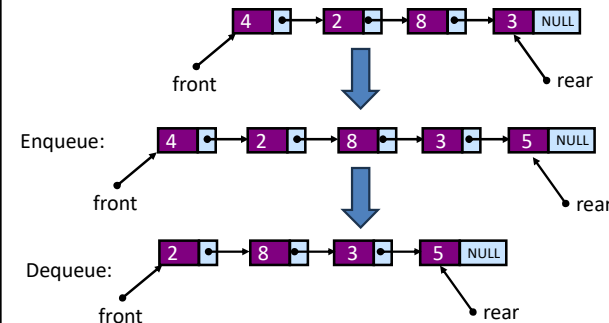
- Option 2: circular queue ["wrap around"]

Implementing a Queue

- Just like a stack, we can implementing a queue in two ways:
 - Using an array
 - Using a linked list**

Implementing a Queue: using linked list

- Store the items in the queue in a linked list
- The top of the queue is the head node, the bottom of the queue is the end of the list
- Enqueue** by adding a new element to the front of the list
- Dequeue** by removing the last element from the list



Queue: Singly linked list Implementation (Python)

```

1st implementation: use class Node as singly linked list only
class Node:
    def __init__(self, data: int):
        self.data = data
        self.next = None
# Check whether the queue is empty
def isEmpty(front):
    return front is None

# Insert a new integer at the rear of the queue
def enqueue(front, rear, value):
    newNode = Node(value)
    if isEmpty(front):
        front = newNode
        rear = newNode
    else:
        rear.next = newNode
        rear = newNode
    return front, rear

# Remove and return the front element of the queue
def dequeue(front, rear):
    if isEmpty(front):
        print("Queue underflow: queue is empty")
        return None, front, rear

    value = front.data
    front = front.next
    if front is None:
        rear = None
    return value, front, rear

# Return the front element without removing it
def frontQ(front):
    if isEmpty(front):
        print("Queue is empty")
        return None
    return front.data

# Return the current number of elements in the queue
def size(front):
    count = 0
    current = front
    while current is not None:
        count += 1
        current = current.next
    return count

# Display all elements of the queue from front to rear
def display(front):
    if isEmpty(front):
        print("Queue is empty")
        return
    current = front
    print("Queue elements from front to rear:")
    while current is not None:
        print(current.data, end=" ")
        current = current.next
    print()

# Example usage
front = None
rear = None
print("Is queue empty?", isEmpty(front))
front, rear = enqueue(front, rear, 10)
front, rear = enqueue(front, rear, 20)
front, rear = enqueue(front, rear, 30)
display(front)
print("Front element:", frontQ(front))
print("Current size:", size(front))
value, front, rear = dequeue(front, rear)
print("Dequeued:", value)
display(front)
print("Front element:", frontQ(front))
print("Is queue empty?", isEmpty(front))
    
```

Queue: Singly linked list Implementation (Python)

```

2nd implementation: use class Node + class Queue
class Node:
    def __init__(self, data: int):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
# Check whether the queue is empty
def isEmpty(self):
    return self.front is None
# Insert a new integer at the rear of the queue
def enqueue(self, value):
    newNode = Node(value)
    if self.isEmpty():
        self.front = newNode
        self.rear = newNode
    else:
        self.rear.next = newNode
        self.rear = newNode
# Remove and return the front element of the queue
def dequeue(self):
    if self.isEmpty():
        print("Queue underflow: queue is empty")
        return None
    value = self.front.data
    self.front = self.front.next
    if self.front is None:
        self.rear = None
    return value
# Return the front element without removing it
def frontQ(self):
    if self.isEmpty():
        print("Queue is empty")
        return None
    return self.front.data

# Return the current number of elements in the queue
def size(self):
    count = 0
    current = self.front
    while current is not None:
        count += 1
        current = current.next
    return count

# Display all elements of the queue from front to rear
def display(self):
    if self.isEmpty():
        print("Queue is empty")
        return
    current = self.front
    print("Queue elements from front to rear:")
    while current is not None:
        print(current.data, end=" ")
        current = current.next
    print()

# Example usage
q = Queue()
print("Is queue empty?", q.isEmpty())
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.display()
print("Front element:", q.frontQ())
print("Current size:", q.size())
value = q.dequeue()
print("Dequeued:", value)
q.display()
print("Front element:", q.frontQ())
print("Is queue empty?", q.isEmpty())
    
```

Queue: Singly linked list Implementation (C)

```
#include <stdio.h>
#include <stdlib.h>
// Define a node of the singly linked list
typedef struct Node {
    int data;
    struct Node* next;
} Node;
// Define the queue
typedef struct Queue {
    Node* front;
    Node* rear;
    int count;
} Queue;
// Initialize the queue
void initQueue(Queue* q) {
    q->front = NULL;
    q->rear = NULL;
    q->count = 0;
}
// Check whether the queue is empty
int isEmpty(Queue* q) {
    return q->front == NULL;
}
// Insert a new integer at the rear of the queue
void enqueue(Queue* q, int value) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        printf("Memory allocation failed\n");
        return;
    }
    newNode->data = value;
    newNode->next = NULL;
    if (isEmpty(q)) {
        q->front = newNode;
        q->rear = newNode;
    } else {
        q->rear->next = newNode;
        q->rear = newNode;
    }
    q->count++;
}
// Remove and return the front element of the queue
int dequeue(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue underflow: queue is empty\n");
        return -1;
    }
    Node* temp = q->front;
    int value = temp->data;
    q->front = q->front->next;
    if (q->front == NULL) q->rear = NULL;
    free(temp);
    q->count--;
    return value;
}
// Return the front element without removing it
int frontQ(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n");
        return -1;
    }
    return q->front->data;
}
// Return the current number of elements in the queue
int size(Queue* q) {
    return q->count;
}
// Display all elements of the queue from front to rear
void display(Queue* q) {
    if (isEmpty(q)) {
        printf("Queue is empty\n"); return;
    }
    Node* current = q->front;
    printf("Queue elements from front to rear:\n");
    while (current != NULL) {
        printf("%d\n", current->data);
        current = current->next;
    }
}
```

Queue: Singly linked list Implementation (C)

```
// Free all nodes in the queue
void freeQueue(Queue* q) {
    Node* current = q->front;
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    q->front = NULL;
    q->rear = NULL;
    q->count = 0;
}

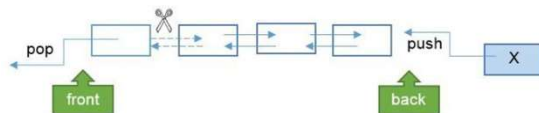
int main() {
    Queue q;
    initQueue(&q);
    printf("Is queue empty? %s\n", isEmpty(&q) ? "Yes" : "No");
    enqueue(&q, 10);
    enqueue(&q, 20);
    enqueue(&q, 30);
    display(&q);
    printf("Front element: %d\n", frontQ(&q));
    printf("Dequeued: %d\n", dequeue(&q));

    display(&q);
    printf("Current size: %d\n", size(&q));
    printf("Is queue empty? %s\n", isEmpty(&q) ? "Yes" : "No");
    freeQueue(&q);

    return 0;
}
```

Library STL in C++: queue

- `#include <queue>` a list type where insertion is performed at the end of the list and removal at the top of the list.



Example of declaration:

`queue<int> s1;` //declare queue s1 in which type of each element is int

- Similar to stack, it is not allowed to declare `queue<int> myQueue (5)` to initialize 5 empty elements.
- Similarly, it is not allowed to declare `queue<int> myQueue (5,100)` to initialize 5 elements in queue with value of 100.

Library in Python: deque

- **deque**: a data structure from the **collections** module where insertion is performed at the end of the sequence and removal is performed at the front of the sequence.

```
from collections import deque
q1 = deque() #declare queue q1
q1.append(x) #insert an element at the rear of the queue
q1.popleft() #remove an element from the front of the queue
q1[0] #get the front element
len(q1) #return the number of elements in the queue
len(q1) == 0 #check whether the queue is empty
```


Library in Python: deque

Example:

```
from collections import deque
q1 = deque()          # declare queue q1
q1.append(10)          # insert 10
q1.append(20)          # insert 20
q1.append(30)          # insert 30

print(q1)              # queue contents
print(q1[0])           # front element
print(len(q1))         # number of elements
print(len(q1) == 0)    # check if empty

q1.popleft()           # remove front element
print(q1)
```

129

Library in Python: queue

- **queue.Queue**: a queue data structure from the queue module where insertion is performed at the rear of the queue and removal is performed at the front of the queue.

```
from queue import Queue
q1 = Queue()           #declare queue q1
q1.put(x)              #insert an element at the rear of the queue
q1.get()               #remove an element from the front of the queue
q1.qsize()             #return the number of elements in the queue
q1.empty()             #check whether the queue is empty
q1[0]                  #get the front element
```

130

Library STL in C++: queue

Capacity	
size()	Return number of elements in the queue
empty()	True/False
Access to an element	
front()	Access to the first element of the queue
back()	Access to the last element of the queue
Query	
push(x)	Insert a new element of value x at the end of the queue. Size of queue will be increased by 1
pop()	Remove the element at the top of queue. Size of queue will be reduced by 1
swap(queue q1, queue q2)	Swap elements of queue q1 and q2

131

Application 1: recognizing palindromes

- A *palindrome* is a string that reads the same forward and backward.

Example: NOON, DEED, RADAR, MADAM

Able was I ere I saw Elba

- How to recognize a given string is a palindrome or not:
 - Step 1: We will put all characters of the string into both a stack and a queue.
 - Step 2: Compare the contents of the stack and the queue character-by-character to see if they would produce the same string of characters:
 - If yes: the given string is a palindrome
 - Otherwise: not palindrome

NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Example 1: Whether "RADAR" is a palindrome or not

Step 1: Put "RADAR" into Queue and Stack:

Current character	Queue (front on the left, rear on the right)	Stack (top on the left)
R	R	R
A	R A	A R
D	R A D	D A R
A	R A D A	A D A R
R	R A D A R	R A D A R
	front rear	top

Example 1: Whether "RADAR" is a palindrome or not

Step 2: Delete "RADAR" from Queue and Stack:

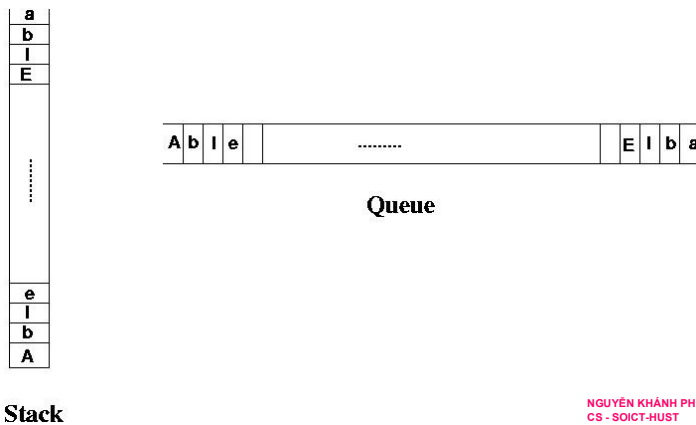
- Dequeue until the queue is empty
- Pop the stack until the stack is empty

Queue (front on the left)	Front of Queue	Top of Stack	Stack (top on the left)
R A D A R	R	R	R A D A R
A D A R	A	A	A D A R
D A R	D	D	D A R
A R	A	A	A R
R	R	R	R
empty	empty	empty	empty

Conclusion: String "RADAR" is a palindrome

Example 2: recognizing palindromes

Able was I ere I saw Elba



NGUYỄN KHÁNH PHƯƠNG
CS - SOICT-HUST

Application 2: Convert a string of digits into a decimal number

The algorithm is described as following:

```
// Convert sequence of digits stored in queue Q into decimal number n
// Remove empty space if any
do { dequeue(Q, ch)
} until ( ch != blank)
// ch is now the first digit of the given string
// Calculate n from sequence of digit in the queue
n = 0;
done = false;
do {
    n = 10 * n + decimal number that ch represents;
    if (! isEmpty(Q))
        dequeue(Q, ch)
    else
        done = true
} until ( done || ch != digit)
// Result: n is the decimal number need to be found
```

NGUYỄN KHÁNH PHƯƠNG 136
CS - SOICT-HUST